

Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia

Relatório do Trabalho Prático

Processamento de Linguagens

Implementação de Analisador e Interpretador para a Linguagem LFun

Licenciatura em Engenharia de Sistemas Informáticos
2.º Ano, 2.º Semestre — 2025/2026

Realizado por:

23510 — Guilherme Alves Azeredo
31528 — Leonardo Kusnierz da Costa
31502 — Rodrigo Henrique Silva Pinheiro

maio de 2026

1. Introdução

Este relatório documenta o trabalho prático da unidade curricular de Processamento de Linguagens, desenvolvido no âmbito da Licenciatura em Engenharia de Sistemas Informáticos (LESI), 2.º ano, 2.º semestre, ano letivo de 2025/2026.

O trabalho centrou-se no desenvolvimento progressivo de um interpretador completo para uma linguagem de programação funcional simplificada denominada LFun (Linguagem Funcional). O interpretador inclui todas as componentes necessárias ao processamento de linguagens: análise léxica, análise sintática, representação intermédia através de uma Árvore de Sintaxe Abstrata (AST) e avaliação com suporte a correspondência de padrões (pattern matching).

O projeto foi estruturado em quatro fases incrementais (A, B, C e D), permitindo uma construção gradual e a consolidação progressiva dos conceitos fundamentais.

2. Objectivos

Os objectivos principais deste trabalho foram:

1. Compreender e aplicar os conceitos teóricos de processamento de linguagens, nomeadamente análise léxica, análise sintática, representação intermédia e avaliação de código.
2. Implementar um analisador léxico capaz de identificar palavras-chave, operadores, literais e comentários da linguagem LFun.
3. Desenvolver um analisador sintático que verifique a conformidade estrutural do código e construa uma Árvore de Sintaxe Abstrata (AST) utilizando classes Python.
4. Criar um interpretador com suporte a funções, condicional if/then/else, seleção de casos com when e verificação de tipos.
5. Garantir qualidade através de testes representativos para cada fase do projeto.

3. Estrutura do Projeto

3.1 Organização Geral

O projeto está organizado em quatro pastas, uma por fase. Cada pasta contém apenas os ficheiros relevantes para essa etapa, sendo o `lfun_lexer.py` e o `ast_nodes.py` partilhados por todas as fases.

Pasta	Objectivo	Ficheiros Principais
A	Análise Léxica	<code>lfun_lexer.py</code> , <code>lfun_lexer_test.py</code>
B	Análise Sintática — Expressões e Let	<code>lfun_lexer.py</code> , <code>ast_nodes.py</code> , <code>lfun_grammar.py</code> , <code>lfun_grammar_test.py</code>
C	Análise Sintática — Funções e If	<code>lfun_lexer.py</code> , <code>ast_nodes.py</code> , <code>lfun_grammar.py</code> , <code>lfun_grammar_test.py</code>
D	Representação Intermédia e Avaliação	<code>lfun_lexer.py</code> , <code>ast_nodes.py</code> , <code>lfun_grammar.py</code> , <code>lfun_eval.py</code> , <code>lfun_main.py</code> , <code>exemplo_entrada.lf</code>

3.2 Arquitetura do Sistema

O sistema segue uma arquitetura em camadas sequenciais, em que cada camada transforma a informação numa forma mais estruturada e útil:

```
Ficheiro .lf (texto)
    ↓
Análise Léxica (lfun_lexer.py) → tokens
    ↓
Análise Sintática (lfun_grammar.py) → AST (objetos Python)
    ↓
Avaliação (lfun_eval.py) → resultado
```

3.2.1 Componentes Principais

O sistema é composto pelos seguintes módulos:

- `lfun_lexer.py` — Analisador léxico. Lê o código fonte e produz tokens.
- `ast_nodes.py` — Classes da AST. Define a representação intermédia do programa.
- `lfun_grammar.py` — Analisador sintático. Verifica a gramática e constrói a AST.
- `lfun_eval.py` — Avaliador. Percorre a AST e calcula os resultados.
- `lfun_main.py` — Programa principal. Aceita ficheiro `.lf` ou modo interativo.

4. Implementação

4.1 Parte A — Análise Léxica

A primeira fase consistiu na implementação do analisador léxico, responsável por ler o código fonte e dividir o texto em tokens — as unidades mínimas com significado na linguagem.

O lexer foi implementado como uma classe Python (LFunLexer) utilizando a biblioteca PLY (ply.lex), seguindo o padrão apresentado nas aulas.

4.1.1 Funcionalidades

- Reconhecimento de palavras reservadas: let, fun, if, then, else, when, is, end, true, false, Int, Bool.
- Identificação de operadores simples (+, -, *, /) e compostos (==, !=, <=, >=, &&, ||, ->).
- Reconhecimento de literais inteiros (NUMBER) e identificadores (ID).
- Ignorar comentários de linha (--) e de bloco ({- ... -}).
- Controlo do número de linha para mensagens de erro precisas.

4.1.2 Estrutura do Lexer

A solução para o problema das palavras reservadas é feita dentro da regra `t_ID`, usando um dicionário `reserved`. Assim, garantimos que palavras como "if" ou "let" nunca são confundidas com nomes de variáveis, mesmo que apareçam no início de um identificador maior como "ifElse":

```
def t_ID(self, t):  
    r'[a-zA-Z][a-zA-Z0-9_]*'  
    # verifica se é palavra reservada  
    t.type = LFunLexer.reserved.get(t.value, 'ID')  
    return t
```

Exemplo de tokenização:

```
Entrada:  let base : Int = 10;  
Tokens:  LET ID(base) ':' TINT '=' NUMBER(10) ';' ;
```

4.2 Parte B — Análise Sintática: Expressões e Let

A segunda fase consistiu na implementação do analisador sintático para expressões aritméticas, booleanas e definições let. O parser foi implementado como uma classe (LFunGrammar) utilizando PLY (ply.yacc).

4.2.1 Gramática

As regras gramaticais implementadas nesta fase foram:

```
Program  : StmtList  
StmtList: Stmt | StmtList Stmt  
Stmt     : E ';' | LET ID ':' Type '=' E ';' ;  
Type     : TINT | TBOOL  
E        : E op E | '-' E | '(' E ')'  
         | NUMBER | TRUE | FALSE | ID
```

4.2.2 Precedência dos Operadores

A precedência é declarada explicitamente para garantir que, por exemplo, $3 + 4 * 2$ é avaliado como $3 + (4 * 2) = 11$ e não como $(3 + 4) * 2 = 14$:

```
precedence = (  
    ('left', 'OR'),          # menor prioridade  
    ('left', 'AND'),  
    ('left', 'EQ', 'NEQ'),  
    ('left', '<', '>', 'LE', 'GE'),  
    ('left', '+', '-'),  
    ('left', '*', '/'),  
    ('right', 'UMINUS'),     # maior prioridade  
)
```

4.2.3 Utilização de PLY e AST com Classes

Em vez de utilizar dicionários Python para representar a AST (como no exemplo da aula), optámos por utilizar classes definidas no ficheiro `ast_nodes.py`. Esta abordagem é mais organizada e legível, pois cada nó tem atributos com nomes claros.

Exemplo de regra gramatical que produz um nó AST:

```
def p_stmt_let_val(self, p):  
    """ Stmt : LET ID ':' Type '=' E ';' """  
    # let x : Int = 3 + 4;  
    p[0] = LetStmt(p[2], p[4], p[6])
```

Aqui `p[2]` é o nome da variável, `p[4]` é o tipo declarado e `p[6]` é a expressão. O resultado é um objeto `LetStmt` com atributos `name`, `type` e `expr`.

4.3 Parte C — Análise Sintática: Funções e If

A terceira fase adicionou suporte a funções e ao condicional `if/then/else`. O ficheiro `lfun_grammar.py` foi expandido com novas regras gramaticais.

4.3.1 Novas Regras Gramaticais

```
Stmt : FUN ID ':' Type ARROW Type ';' -- assinatura  
      | LET ID ID '=' E ';'           -- definição  
E    : IF E THEN E ELSE E             -- condicional  
      | ID '(' E ')'                  -- chamada de função
```

4.3.2 Exemplos

Assinatura e definição de função:

```
fun dobro : Int -> Int;  
let dobro n = n * 2;  
dobro(7);    -- resultado: 14 tipo: Int
```

Função com condicional:

```
fun absoluto : Int -> Int;  
let absoluto n = if n < 0 then 0 - n else n;  
absoluto(-5);    -- resultado: 5 tipo: Int  
absoluto(8);     -- resultado: 8 tipo: Int
```

4.4 Parte D — Representação Intermédia e Avaliação

A quarta fase introduziu o ficheiro `ast_nodes.py` com as classes da AST, o avaliador (`lfun_eval.py`) e o programa principal (`lfun_main.py`). Foi também adicionado suporte ao `when`.

4.4.1 Nós da AST (`ast_nodes.py`)

Cada construção da linguagem é representada por uma classe Python:

Classe	Representa	Atributos
NumberLiteral	Número inteiro	value
BoolLiteral	Valor booleano	value
VarExpr	Referência a variável	name
BinOp	Operação binária (E op E)	op, left, right
UnaryOp	Operação unária (-E)	op, operand
IfExpr	Condicional if/then/else	cond, then_e, else_e
CallExpr	Chamada de função f(arg)	name, arg
MatchExpr	Seleção when (expr) is...end	expr, cases
LetStmt	Definição let x : T = E	name, type, expr
FunSig	Assinatura fun f : T -> T	name, arg_type, ret_type
FunDef	Definição let f x = E	name, arg, body
ExprStmt	Instrução de expressão E;	expr
Program	Programa completo	stmts
IntPattern	Padrão inteiro no when	value
BoolPattern	Padrão booleano no when	value
WildcardPattern	Padrão _ (qualquer valor)	—

4.4.2 Pattern Matching com When

O `when` permite seleccionar um resultado em função do valor de uma expressão. Cada caso tem um padrão e um resultado. O avaliador testa os padrões por ordem e devolve o resultado do primeiro que corresponder. O wildcard (`_`) corresponde sempre a qualquer valor.

Padrões suportados:

- Literais inteiros: 0, 1, -1, ...
- Literais booleanos: true, false
- Wildcard: `_` (caso contrário, corresponde a qualquer valor)

Exemplo:

```

fun eZero : Int -> Bool;
let eZero n =
  when (n) is
    0 -> true;
    _ -> false;
end;
eZero(0);      -- resultado: True tipo: Bool
eZero(5);      -- resultado: False tipo: Bool

```

4.4.3 Avaliador (lfun_eval.py)

O avaliador percorre a AST recursivamente, tratando cada tipo de nó de forma específica. Segue o padrão da aula, com tabelas de variáveis e funções e um dicionário de operadores:

```

symbols = {} # variáveis: { 'x': 10, 'area': 20 }
functions = {} # funções: { 'dobro': ('n', corpo) }
operators = {
  '+': lambda a, b: a + b,
  '-': lambda a, b: a - b,
  '*': lambda a, b: a * b,
  '==': lambda a, b: a == b,
  # ...
}

```

O avaliador também verifica se o tipo da expressão corresponde ao tipo declarado no let. Por exemplo, "let x : Int = true;" produz um erro porque true é do tipo Bool e não Int.

4.4.4 Programa Principal (lfun_main.py)

O lfun_main.py suporta dois modos de execução, seguindo o padrão do arith_main.py das aulas:

- Modo ficheiro: py lfun_main.py exemplo_entrada.lf — lê e executa o ficheiro completo.
- Modo interativo: py lfun_main.py — lê e executa instrução a instrução, como uma consola.

5. Representação Intermédia — ast_nodes.py

O ficheiro `ast_nodes.py` é partilhado por todas as fases do projeto (B, C e D). Define todas as classes necessárias para representar qualquer construção da linguagem LFun, incluindo os padrões do `when`.

A decisão de usar classes em vez de dicionários (como no exemplo da aula) permite um código mais organizado e legível. Cada classe tem atributos com nomes claros (por exemplo, `node.name`, `node.body`, `node.cond`) em vez de posições num dicionário (`dict['args'][0]`).

Esta abordagem facilita também a deteção de erros — se tentarmos aceder a um atributo que não existe, Python dá um erro claro, ao contrário de um dicionário que simplesmente devolve `None`.

6. Tecnologias Utilizadas

Tecnologia	Utilização
Python 3.14	Linguagem de implementação principal
PLY 3.11 (Python Lex-Yacc)	Biblioteca para construção do analisador léxico (ply.lex) e sintático (ply.yacc)
ply.lex	Motor do analisador léxico — reconhece padrões com expressões regulares
ply.yacc	Motor do analisador sintático — implementa o algoritmo LALR
VS Code	Ambiente de desenvolvimento
Git / GitHub	Controlo de versões e partilha de código

7. Testes e Validação

7.1 Estratégia de Testes

Cada fase tem um ficheiro de testes dedicado que cobre os casos relevantes para essa etapa. Os testes são executados diretamente no terminal e o output é comparado visualmente com o esperado.

7.2 Testes por Fase

Fase A — Ifun_lexer_test.py

Testa o reconhecimento correto de todos os tokens da linguagem:

- Expressões aritméticas e booleanas.
- Palavras reservadas (let, fun, if, when, true, false, Int, Bool).
- Operadores compostos (==, !=, <=, >=, &&, ||, ->).
- Comentários de linha e de bloco (devem ser ignorados).
- Construção when com wildcard (_).

Exemplo de output:

```
frase: 'let base : Int = 10;'  
tokens: LET ID : TINT = NUMBER ;  
  
frase: '-- comentário de linha  
let x : Int = 1;'  
tokens: LET ID : TINT = NUMBER ;   ← comentário ignorado
```

Fase B — Ifun_grammar_test.py

Testa a construção da AST para expressões e definições let:

- Expressões aritméticas com precedência correta: $10 - 2 * 3 = 4$ e não 24.
- Expressões booleanas: $3 < 5$, $10 == 7$.
- Definições let com tipo Int e Bool.
- Utilização de variáveis em expressões: largura * altura.

Fase C — Ifun_grammar_test.py

Testa a construção da AST para funções e condicionais:

- Assinatura de função: fun dobro : Int -> Int.
- Definição de função com um argumento: let dobro n = n * 2.
- Chamada de função: dobro(7).
- Função com condicional if: let absoluto n = if n < 0 then 0 - n else n.
- Menos unário: absoluto(-5).

Fase D — Ifun_main.py

Testa a execução completa do interpretador com o ficheiro exemplo_entrada.lf:

Instrução	Resultado Esperado	Resultado Obtido
3 + 4;	resultado: 7 tipo: Int	✓
10 - 2 * 3;	resultado: 4 tipo: Int	✓
let base : Int = 10;	(sem output — definição)	✓
dobro(7);	resultado: 14 tipo: Int	✓
ePositivo(3);	resultado: True tipo: Bool	✓
ePositivo(-2);	resultado: False tipo: Bool	✓
absoluto(-5);	resultado: 5 tipo: Int	✓
absoluto(8);	resultado: 8 tipo: Int	✓
eZero(0);	resultado: True tipo: Bool	✓
eZero(5);	resultado: False tipo: Bool	✓
boolToInt(true);	resultado: 1 tipo: Int	✓
boolToInt(false);	resultado: 0 tipo: Int	✓

8. Resultados e Demonstração

8.1 Modo Ficheiro

Execução do interpretador com o ficheiro exemplo_entrada.lf:

```
| py lfun_main.py exemplo_entrada.lf
```

Output produzido:

```
resultado: 7 tipo: Int
resultado: 4 tipo: Int
resultado: 14 tipo: Int
resultado: True tipo: Bool
resultado: False tipo: Bool
resultado: 5 tipo: Int
resultado: 8 tipo: Int
resultado: True tipo: Bool
resultado: False tipo: Bool
resultado: 1 tipo: Int
resultado: 0 tipo: Int
```

8.2 Modo Interativo

O interpretador pode também ser utilizado em modo interativo:

```
py lfun_main.py
>> fun dobro : Int -> Int;
>> let dobro n = n * 2;
>> dobro(7);
resultado: 14 tipo: Int
>> let x : Int = true;
Erro de tipo: 'x' declarado como Int mas a expressão tem tipo Bool
```

8.3 Conformidade com Requisitos

Requisito do Enunciado	Estado	Evidência
Analisador léxico com ply.lex	Implementado	lfun_lexer.py, lfun_lexer_test.py
Palavras reservadas e comentários	Implementado	Dicionário reserved, t_COMMENT_LINE, t_COMMENT_BLOCK
Analisador sintático com ply.yacc	Implementado	lfun_grammar.py (fases B, C, D)
Expressões aritméticas e booleanas	Implementado	Fase B — grammar_test.py
Definições let com tipo explícito	Implementado	Fase B — LetStmt
Funções com assinatura e definição	Implementado	Fase C — FunSig, FunDef

Condicional if/then/else	Implementado	Fase C — IfExpr
Seleção de casos when	Implementado	Fase D — MatchExpr, Case, padrões
Árvore de Sintaxe Abstrata (AST)	Implementado	ast_nodes.py — classes Python
Verificação de tipos	Implementado	lfun_eval.py — _verificar_tipo
Avaliação e execução	Implementado	lfun_eval.py, lfun_main.py
Modo ficheiro e modo interativo	Implementado	lfun_main.py

9. Conclusão

9.1 Aprendizagens Principais

Este trabalho permitiu consolidar vários conceitos fundamentais de processamento de linguagens:

1. Separação de responsabilidades: cada módulo tem um papel bem definido — o lexer tokeniza, o parser valida e constrói a AST, e o avaliador executa.
2. Representação intermédia: a utilização de classes Python para a AST, em vez de dicionários, tornou o código mais organizado, legível e fácil de estender.
3. Precedência de operadores: a declaração explícita de precedência no PLY resolve automaticamente as ambiguidades gramaticais.
4. Correspondência de padrões (pattern matching): a implementação do when exigiu a criação de classes de padrão específicas e uma função de correspondência genérica.
5. Verificação de tipos: a verificação em tempo de execução garante que os tipos declarados no let correspondem ao tipo real da expressão avaliada.

9.2 Funcionalidades Implementadas

- Análise léxica completa com suporte a comentários de linha e de bloco.
- Parser robusto com precedência declarada e tratamento de erros.
- Representação intermédia genérica com classes Python (AST).
- Avaliador recursivo com tabela de símbolos e tabela de funções.
- Suporte a funções com assinatura de tipo e definição com argumento.
- Condicional if/then/else com verificação de tipo da condição.
- Seleção de casos when com padrões inteiros, booleanos e wildcard.
- Verificação de tipos nas definições let.
- Modo ficheiro e modo interativo no programa principal.

9.3 Possíveis Melhorias Futuras

- Suporte a funções recursivas e funções de ordem superior.
- Tipos adicionais: Char, String, listas e pares.
- Mensagens de erro mais detalhadas com indicação da linha e coluna.
- Geração de código C a partir da AST.
- Verificação de tipos estática (antes da avaliação).

9.4 Reflexão Final

O projeto foi desenvolvido de forma incremental, o que permitiu consolidar os conceitos de cada fase antes de avançar para a seguinte. A estrutura em camadas (lexer → parser → AST → eval) provou ser uma abordagem sólida e facilmente extensível.

A decisão de utilizar classes Python para a AST, em vez de dicionários simples como no exemplo da aula, foi uma opção consciente que tornou o código mais claro e organizado, embora exija um ficheiro adicional (`ast_nodes.py`).

Apêndice A — Estrutura de Ficheiros

Organização completa do repositório do projeto:

```
TP_PL202526/
├── A/
│   ├── lfun_lexer.py           ← analisador léxico
│   └── lfun_lexer_test.py      ← testes do lexer
├── B/
│   ├── lfun_lexer.py
│   ├── ast_nodes.py           ← classes da AST
│   ├── lfun_grammar.py        ← parser: expressões + let
│   └── lfun_grammar_test.py    ← testes da fase B
├── C/
│   ├── lfun_lexer.py
│   ├── ast_nodes.py
│   ├── lfun_grammar.py        ← parser: + funções + if
│   └── lfun_grammar_test.py    ← testes da fase C
└── D/
    ├── lfun_lexer.py
    ├── ast_nodes.py
    ├── lfun_grammar.py        ← parser: + when
    ├── lfun_eval.py           ← avaliador
    ├── lfun_main.py           ← programa principal
    └── exemplo_entrada.lf      ← ficheiro de teste
```

Apêndice B — Repositório GitHub

O código fonte, histórico de versões e documentação complementar deste trabalho estão disponíveis publicamente no GitHub:

`https://github.com/azeredo99/TP\_PL202526`